



INTRODUÇÃO A MICROCONTROLADORES

Guia de referência para projetos
práticos com C++.

Autor

Fábio Ribeiro Filho

Engenharia da Computação

UFPA / ITEC



1 Fundamentos de C++: Variáveis e Constantes

Antes de acionar componentes físicos, é necessário armazenar informações na memória do microcontrolador. Usar nomes descritivos e constantes facilita a organização do código e evita alterações acidentais nos pinos.

```
1 // Definicao de Pinos
2 const int pinoLed = 13;    // 'const' garante que o valor nunca sera
    alterado
3 #define pinoBotao 2
4
5 // Variaveis Comuns
6 int contador = 0;          // Numeros inteiros
7 float temperatura = 25.5;  // Numeros com casas decimais
8 bool ligado = true;        // Variavel logica (true ou false)
```

2 Estrutura Geral do Código

Nos projetos que vamos desenvolver, utilizaremos duas funções principais:

```
1 void setup() {
2     // Executa apenas UMA vez quando o microcontrolador liga
3     // Usado para configuracoes, definir modos dos pinos e iniciar
    comunicacao
4 }
5
6 void loop() {
7     // Executa repetidamente em um ciclo infinito
8     // Aqui fica a logica principal (ler botoes, piscar LEDs, etc)
9 }
```

3 Guia de Funções (Input/Output e Tempo)

Abaixo estão os comandos essenciais para interagir com os componentes eletrônicos através dos pinos:

- `pinMode()`: Configura se o pino vai enviar (OUTPUT) ou receber (INPUT) sinais.

```
pinMode(pinoLed, OUTPUT);
```

- `digitalWrite()`: Envia nível lógico alto (HIGH) ou baixo (LOW) para um pino.

```
digitalWrite(pinoLed, HIGH);
```

- `digitalRead()`: Lê se o pino está recebendo tensão, retornando HIGH ou LOW.

```
int est = digitalRead(2);
```

- `analogWrite()`: Simula uma saída analógica (PWM). O valor varia de 0 a 255.

```
analogWrite(pinoLed, 127);
```

- `analogRead()`: Lê valores analógicos (0 a 1023 no Arduino ou até 4095 no ESP32).

```
int luz = analogRead(A0);
```

- `delay()`: Pausa o processamento pelo tempo especificado em milissegundos.

```
delay(500);
```

4 Comunicação Serial

Para debugar o código ou visualizar os dados lidos pelos sensores no computador, utilizamos o Monitor Serial.

```
1 void setup() {
2     Serial.begin(9600); // Define a velocidade de 9600 bps; taxas
      maiores aceleram o envio.
3 }
4
5 void loop() {
6     int valorSensor = 42;
7     Serial.print("0 valor lido eh: "); // Imprime o texto sem quebrar
      a linha
8     Serial.println(valorSensor);      // Imprime o valor e pula para
      a proxima linha
9
10    delay(1000); // Aguarda 1 segundo (1000 milissegundos) antes de
      repetir
11 }
```